

INFORME TÉCNICO: IMPLEMENTACIÓN Y ANÁLISIS COMPARATIVO DE ALGORITMOS GRÁFICOS BIDIMENSIONALES EN C# WINDOWS FORMS

Asignatura: Computación Gráfica

Nombre: Josue Andel Chango Parra

ÍNDICE DE CONTENIDOS

- 1. CAPÍTULO 1: INTRODUCCIÓN Y OBJETIVOS
 - 1.1. Introducción
 - 1.2. Objetivos del Proyecto
- 2. CAPÍTULO 2: ALGORITMOS DE TRAZADO DE LÍNEAS
 - 2.1. Algoritmo DDA (Digital Differential Analyzer)
 - 2.2. Algoritmo de Bresenham para Líneas
 - 2.3. Algoritmo del Punto Medio para Líneas
- 3. CAPÍTULO 3: ALGORITMOS DE TRAZADO DE CIRCUNFERENCIAS
 - 3.1. Algoritmo de Circunferencia mediante Punto Medio
 - 3.2. Algoritmo de Circunferencia de Bresenham
 - 3.3. Algoritmo de Circunferencia Paramétrica
- 4. CAPÍTULO 4: ALGORITMOS DE RELLENO DE REGIONES
 - 4.1. Algoritmo Flood Fill (Relleno de Inundación)
 - 4.2. Algoritmo Boundary Fill (Relleno de Frontera)
 - 4.3. Algoritmo Scanline Flood Fill
- 5. CAPÍTULO 5: IMPLEMENTACIÓN DE ANIMACIONES MEDIANTE TIMERS
 - 5.1. Mecanismo de Animación Paso a Paso
 - 5.2. Control de Flujo y Sincronización
- 6. CAPÍTULO 6: TABLAS COMPARATIVAS DE RENDIMIENTO Y PRECISIÓN
- 7. CAPÍTULO 7: CONCLUSIONES

CAPÍTULO 1: INTRODUCCIÓN Y OBJETIVOS

1.1. Introducción

El desarrollo de la computación gráfica moderna se sustenta sobre pilares algorítmicos clásicos que determinan la manera en que los entornos abstractos de software se traducen en representaciones matriciales de píxeles en una pantalla. El presente informe técnico detalla el diseño, la implementación y la evaluación de un sistema de software desarrollado en el lenguaje C# bajo el entorno de desarrollo de interfaces Windows Forms. Este sistema reúne las técnicas fundamentales de discretización geométrica y manipulación topológica de regiones divididas en tres categorías esenciales: trazado de líneas rectilíneas, generación de curvas circulares y algoritmos de relleno espacial.

La particularidad de esta implementación radica en que trasciende la mera ejecución estática de los algoritmos; el sistema integra un motor de animación temporal que expone la progresión interna de cada método elemental en tiempo real. A través de esta aproximación, se visibiliza el orden y la lógica geométrica con la que el procesador toma decisiones de iluminación sobre la matriz de rasterización.

1.2. Objetivos del Proyecto

- Comprender el funcionamiento interno: Analizar desde una perspectiva matemática y lógica el comportamiento metodológico de cada algoritmo gráfico elemental implementado.
- Analizar diferencias operacionales: Evaluar de qué manera las aproximaciones continuas y discretas afectan el procesamiento y la fidelidad de las figuras geométricas complejas.
- Comparar rendimiento y precisión: Establecer criterios cuantitativos y cualitativos relativos a la carga computacional, uso de memoria y fidelidad geométrica de las técnicas de trazado y relleno.
- Visualizar la lógica de ejecución: Adaptar los algoritmos tradicionales para exhibir interactivamente el orden de generación de píxeles mediante un control secuencial interactivo.

CAPÍTULO 2: ALGORITMOS DE TRAZADO DE LÍNEAS

2.1. Algoritmo DDA (Digital Differential Analyzer)

Fundamento Matemático y Procedimiento:

El analizador diferencial digital (DDA) es un algoritmo de conversión por rasterización que se basa en la evaluación de la ecuación diferencial de la línea recta. Utiliza la pendiente de la línea para calcular de forma incremental las coordenadas de los píxeles adyacentes.

En cada paso recursivo, la posición actual de la línea se actualiza sumando recursivamente incrementos fijos basados en las proporciones de dx y dy . Debido a que los incrementos

operan bajo el dominio de los números reales (punto flotante), las coordenadas obtenidas no corresponden directamente a posiciones discretas de la cuadrícula de pantalla. Por lo tanto, el algoritmo requiere la aplicación explícita de una función de redondeo numérico para proyectar el punto flotante al entero más cercano antes de pintar el píxel correspondiente en la interfaz.

Ventajas: Es un algoritmo sumamente intuitivo de implementar y comprender matemáticamente. Garantiza una distribución uniforme de píxeles a lo largo del trayecto debido al cálculo basado en el diferencial máximo.

Desventajas: El uso intrínseco de operaciones en punto flotante para los incrementos horizontales y verticales reduce la velocidad de ejecución en hardware que carece de unidades de optimización dedicadas. Adicionalmente, la operación de redondeo acumulada en cada ciclo añade un coste computacional crítico por cada píxel impreso.

2.2. Algoritmo de Bresenham para Líneas

Funcionamiento Interno y Parámetro de Decisión:

El algoritmo de líneas desarrollado por Bresenham representa una optimización matemática sustancial frente al DDA, dado que elimina por completo la dependencia de la aritmética de punto flotante. Su estrategia se fundamenta en el análisis del signo de un parámetro de decisión entero, el cual mide la desviación del trazo geométrico ideal respecto a los centros de los píxeles reales candidatos de la pantalla.

Actualización del Error y Ventajas frente a DDA:

En cada iteración, se evalúa el signo algebraico de la variable de decisión. Al prescindir de divisiones, punto flotante y redondeos continuos, el algoritmo de Bresenham puede ejecutarse enteramente mediante adiciones aritméticas rápidas y operaciones de desplazamiento de bits a nivel de hardware. Esto se traduce en una eficiencia temporal significativamente mayor y un uso óptimo del ciclo de reloj del procesador central.

2.3. Algoritmo del Punto Medio para Líneas

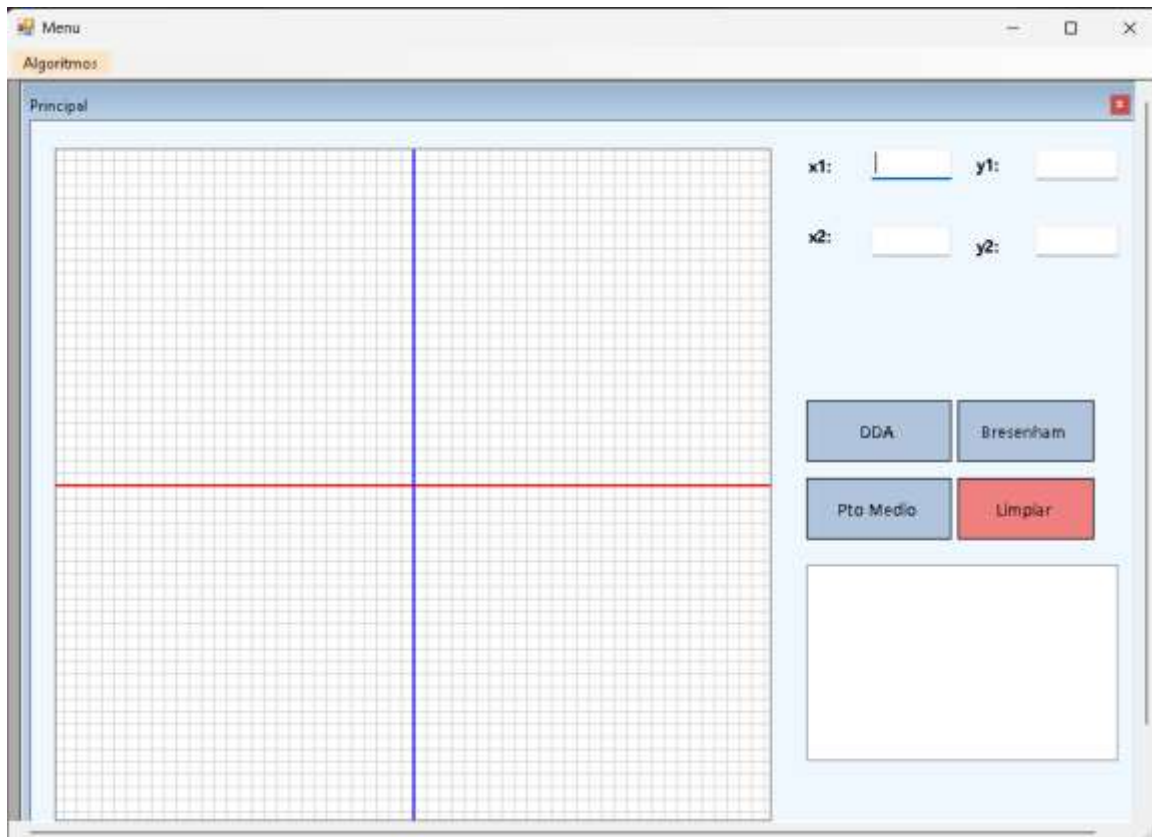
Parámetro de Decisión y Selección de Píxeles:

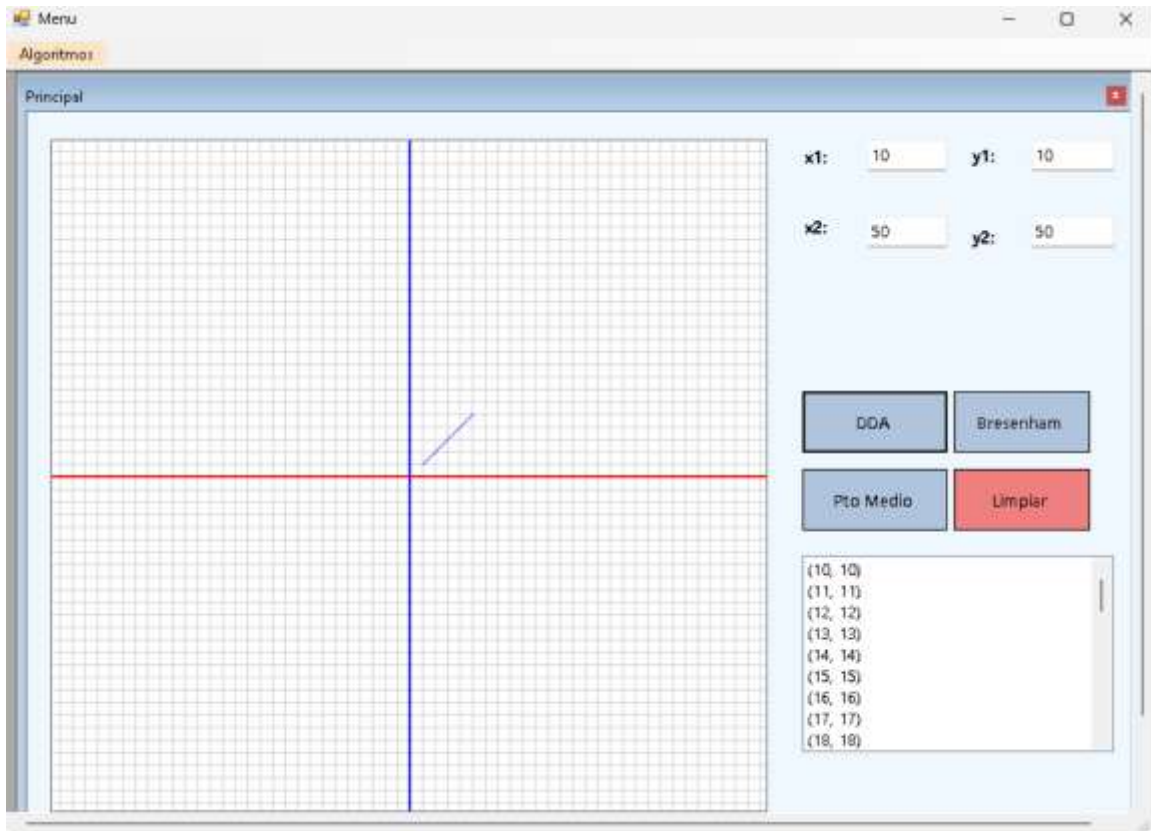
El algoritmo del Punto Medio para líneas aborda la discretización geométrica reformulando el problema mediante la evaluación de la función implícita de la recta. Al trazar la línea entre dos puntos, el algoritmo decide el siguiente píxel evaluando el signo de esta función en el punto medio exacto situado entre las dos ubicaciones físicas de los píxeles candidatos más próximos.

Relación con Bresenham y Diferencias respecto a DDA:

Existe una equivalencia matemática directa entre el método del Punto Medio y el algoritmo clásico de Bresenham para líneas rectilíneas. A diferencia del DDA, que requiere el almacenamiento y cómputo iterativo de variables numéricas reales de alta precisión para

evitar la acumulación excesiva de errores por redondeo, el Punto Medio se reduce a un esquema puramente entero.





CAPÍTULO 3: ALGORITMOS DE TRAZADO DE CIRCUNFERENCIAS

3.1. Algoritmo de Circunferencia mediante Punto Medio

Simetría Octogonal y Parámetro de Decisión:

La generación eficiente de curvas circulares en pantallas matriciales aprovecha las propiedades de simetría geométrica de la figura. El algoritmo del Punto Medio para circunferencias se diseña analizando un único octante de la curva (correspondiente a un ángulo de 45 grados) y proyectando matemáticamente cada píxel calculado hacia las siete regiones restantes de forma inmediata. Esta propiedad de simetría octogonal implica que por cada operación lógica realizada en el ciclo primario, se imprimen simultáneamente ocho píxeles en coordenadas reflejadas de los cuadrantes cartesianos.

3.2. Algoritmo de Circunferencia de Bresenham

Formulación de Decisiones y Rendimiento Computacional:

El algoritmo de circunferencias de Bresenham plantea un enfoque fundamentado en la minimización directa del error cuadrático de la distancia entre los píxeles de la cuadrícula y el radio verdadero de la curva ideal. A diferencia del punto medio convencional, este método deriva de forma explícita un parámetro de control inicial gobernado por operaciones enteras.

Su rendimiento computacional es óptimo debido a que las operaciones críticas dentro del bucle principal se reducen a adiciones ordinarias y multiplicaciones por potencias elementales de dos, las cuales se resuelven de forma instantánea a nivel del procesador de ejecución.

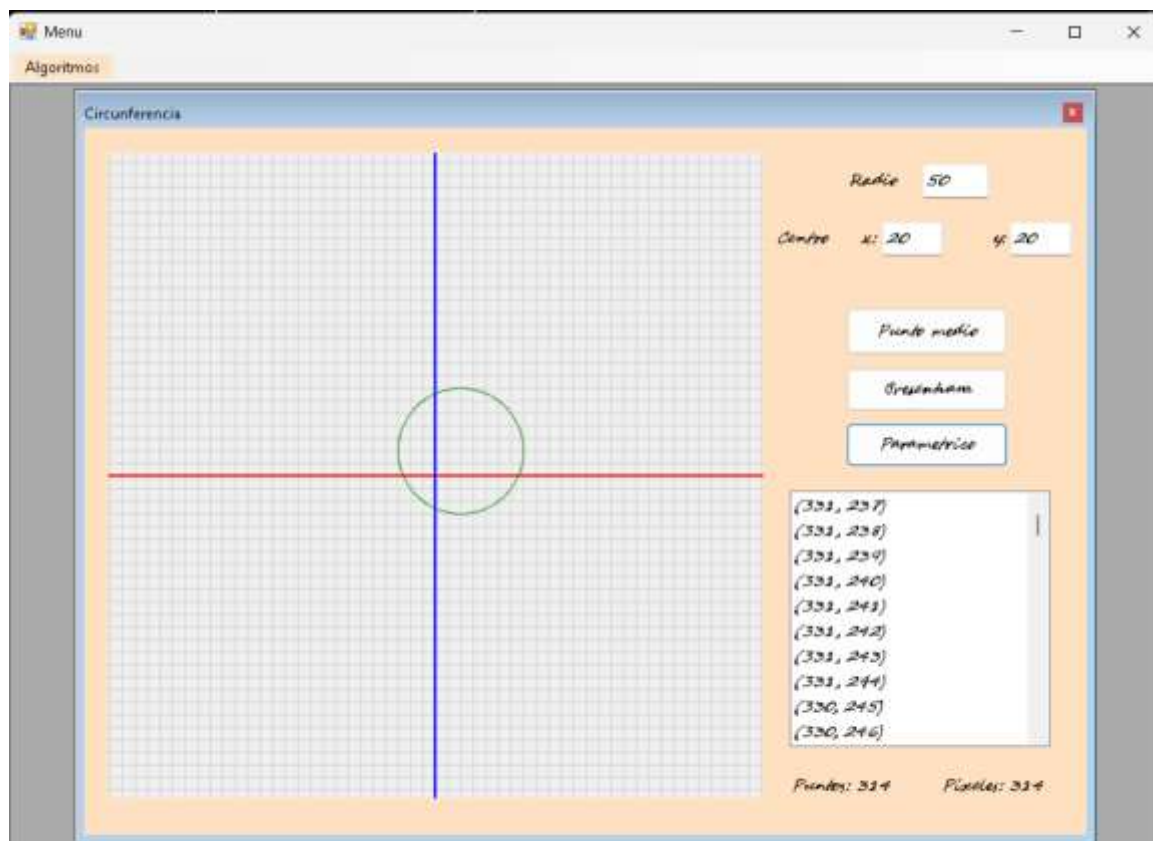
3.3. Algoritmo de Circunferencia Paramétrica

Fundamento Trigonométrico y Ecuaciones de Generación:

La aproximación analítica clásica para generar contornos circulares se basa en la representación trigonométrica polar de la geometría euclidiana. Este método genera la secuencia continua de coordenadas espaciales calculando de forma secuencial las proyecciones horizontales y verticales en base a un ángulo de barrido que varía incrementalmente.

Desventajas Computacionales:

Desde el punto de vista computacional, es el algoritmo de circunferencias menos eficiente. La llamada sistemática a funciones trascendentes de librerías numéricas (seno, coseno) en cada iteración implica operaciones complejas a nivel de CPU. Adicionalmente, el procesamiento requiere transformaciones constantes de números de punto flotante a enteros de coordenadas discretas, ralentizando significativamente el motor gráfico.



CAPÍTULO 4: ALGORITMOS DE RELLENO DE REGIONES

4.1. Algoritmo Flood Fill (Relleno de Inundación)

Mecanismo de Expansión por Vecinos mediante Estructura Stack:

El algoritmo de Relleno de Inundación (Flood Fill) opera de manera homóloga a la propagación de fluidos sobre superficies delimitadas. Su propósito consiste en sustituir uniformemente el color de una región interior conectada que comparte un mismo color de fondo inicial. El proceso comienza a partir de un píxel semilla seleccionado de manera arbitraria dentro del contorno geométrico.

Para evitar el desbordamiento de la pila de llamadas del sistema (Stack Overflow) inherente a las implementaciones recursivas tradicionales sobre áreas de gran tamaño, este sistema utiliza una estructura de datos explícita orientada a objetos de tipo pila (Stack).

4.2. Algoritmo Boundary Fill (Relleno de Frontera)

Principio Operacional y Dependencia de Fronteras:

El algoritmo de Relleno de Frontera (Boundary Fill) comparte similitudes con la aproximación del Flood Fill en cuanto a su modelo de conectividad espacial. Sin embargo, difiere fundamentalmente en su criterio de parada lógica. En lugar de evaluar y propagarse sobre un color de fondo común, este método continúa su expansión radial hasta que encuentra un color de borde específico que delimita la figura geométrica.

Este algoritmo presenta una dependencia absoluta respecto a la integridad cromática de la frontera. Si el contorno perimetral exhibe una discontinuidad de un solo píxel o una variación sutil en el tono del color de borde, el proceso se filtrará fuera de los límites geométricos previstos.

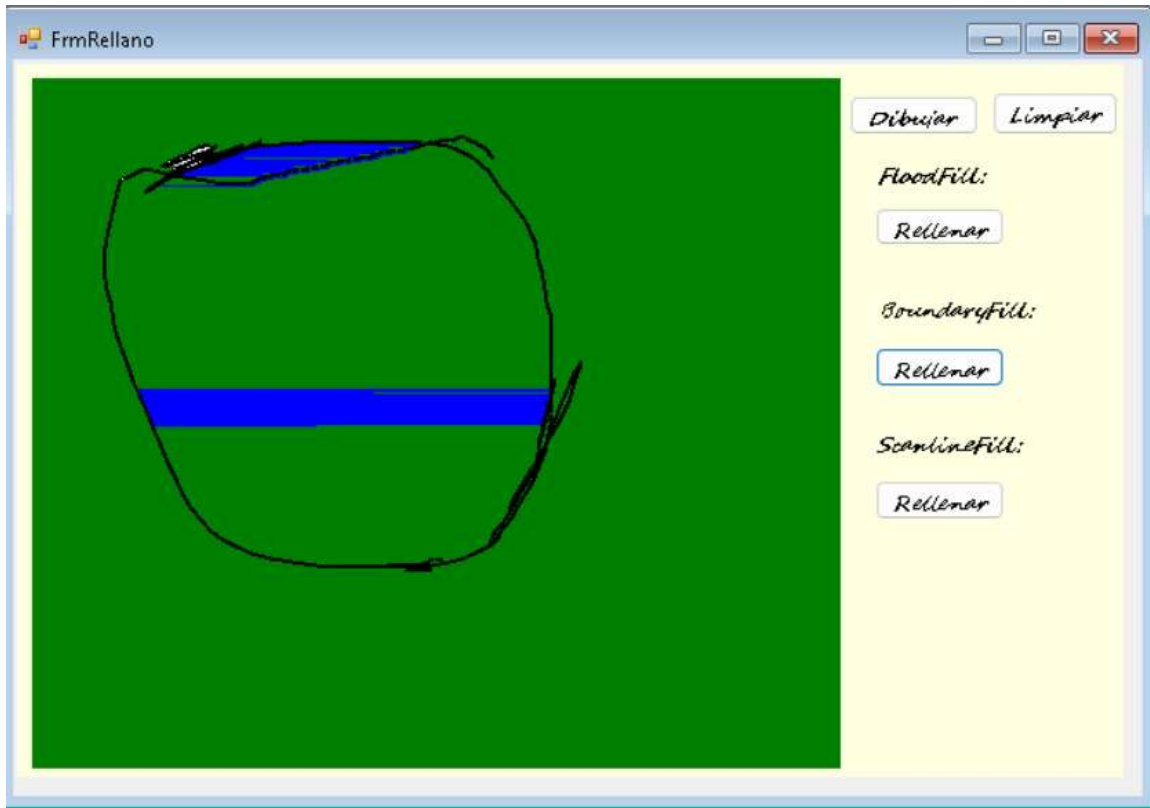
4.3. Algoritmo Scanline Flood Fill

Optimización por Líneas y Reducción del Coste Operacional:

El algoritmo Scanline Flood Fill representa una optimización avanzada orientada a mitigar las ineficiencias de memoria asociadas a los métodos orientados a píxeles individuales. En lugar de evaluar e insertar de forma independiente cada uno de los vecinos de un píxel en la pila, este enfoque procesa segmentos horizontales completos (líneas de escaneo) de manera secuencial.

Ventajas de Rendimiento:

Al insertar únicamente un píxel de referencia por cada segmento horizontal en lugar de registrar cada elemento de forma individual, se reduce notablemente el número de operaciones Push y Pop, y disminuye de forma drástica el tamaño máximo alcanzado por la estructura de almacenamiento temporal en memoria volátil.



CAPÍTULO 5: IMPLEMENTACIÓN DE ANIMACIONES MEDIANTE TIMERS

5.1. Mecanismo de Animación Paso a Paso

Una de las funcionalidades centrales de este proyecto es la capacidad de visualizar dinámicamente y paso a paso el proceso de generación de píxeles de cada algoritmo en la interfaz gráfica de usuario. Para lograr este comportamiento sin bloquear el hilo principal de la interfaz (UI Thread) y permitir una visualización fluida, se diseñó una arquitectura desacoplada basada en componentes Timer de Windows Forms.

5.2. Control de Flujo y Sincronización

La separación funcional entre la lógica matemática y la visualización ofrece ventajas técnicas significativas. Los algoritmos calculan la totalidad de la geometría antes de iniciar la animación, almacenando los resultados en colecciones secuenciales estructuradas. Esto garantiza que los tiempos de cómputo geométrico no interfieran con la fluidez de la animación visual.

CAPÍTULO 6: TABLAS COMPARATIVAS DE RENDIMIENTO Y PRECISIÓN

Tabla 1: Evaluación Comparativa de Algoritmos de Líneas

Criterio de Análisis	Algoritmo DDA	Algoritmo de	Algoritmo del
----------------------	---------------	--------------	---------------

		Bresenham	Punto Medio
Precisión Geométrica	Alta (Sujeta a redondeo)	Absoluta (Matemática ideal)	Absoluta (Matemática ideal)
Velocidad de Cómputo	Media-Baja	Muy Alta (Óptima en hardware)	Muy Alta (Óptima en hardware)
Uso de Enteros	No (Solo coordenadas finales)	Sí (Exclusivo en todo el ciclo)	Sí (Exclusivo en todo el ciclo)
Uso de Punto Flotante	Sí (Incrementos flotantes)	No (Eliminado por completo)	No (Eliminado por completo)
Complejidad de Código	Baja (Muy simple e intuitivo)	Media (Requiere control de octantes)	Media (Ecuación implícita de recta)

Tabla 2: Evaluación Comparativa de Algoritmos de Circunferencias

Criterio de Análisis	Método del Punto Medio	Método de Bresenham	Método Paramétrico
Precisión Estructural	Alta (Aproximación entera)	Alta (Minimiza error cuadrático)	Variable (Depende del $d\theta$)
Velocidad de Respuesta	Alta (Usa sumas enteras)	Alta (Usa sumas enteras)	Muy Baja (Llamadas repetitivas)
Complejidad Lógica	Media (Manejo de 8 octantes)	Media (Manejo de 8 octantes)	Baja (Ecuaciones directas simples)
Uso de Trigonometría	No (Ecuaciones algebraicas)	No (Ecuaciones algebraicas)	Sí (Uso estricto de sin/cos)
Carga Computacional	Muy Reducida	Muy Reducida	Muy Elevada (Cómputo en CPU)

Tabla 3: Evaluación Comparativa de Algoritmos de Relleno de Regiones

Criterio de Análisis	Algoritmo Flood Fill	Algoritmo Boundary Fill	Algoritmo Scanline Flood Fill
Velocidad del Relleno	Media (Evaluación por píxel)	Media (Evaluación por píxel)	Alta (Segmentos horizontales)
Consumo de Memoria	Alto (Pila densa por puntos)	Alto (Pila densa por puntos)	Muy Reducido (Pila optimizada)
Complejidad Operativa	Baja (Lógica secuencial simple)	Baja (Lógica secuencial simple)	Alta (Cálculo bidireccional)
Escalabilidad Espacial	Limitada en áreas masivas	Limitada (Riesgo de fugas)	Excelente en altas resoluciones

CAPÍTULO 7: CONCLUSIONES

Conclusiones Técnicas

- Optimización mediante Aritmética Entera: La sustitución de la aritmética de punto flotante por operaciones puramente enteras representa un principio fundamental de la computación gráfica clásica que maximiza la velocidad de rasterización y optimiza el uso de los ciclos de reloj del hardware de procesamiento.
- Eficiencia a través de la Simetría Geométrica: El aprovechamiento de propiedades geométricas inherentes a las figuras, como la simetría octogonal en el trazado de circunferencias, permite reducir los costes de cálculo matemático en un factor de hasta ocho veces, demostrando que un diseño algorítmico inteligente es más determinante para el rendimiento que la potencia bruta del procesador.
- Limitaciones Computacionales de Métodos Analíticos: El uso de ecuaciones paramétricas basadas en funciones trigonométricas trascendentes resulta ineficiente para la conversión por rasterización en tiempo real debido a su elevado consumo computacional en la CPU, quedando relegado principalmente a tareas de modelado geométrico teórico.